German University in Cairo
Media Engineering and Technology
Prof. Dr. Slim Abdennadher
Dr. Nourhan Ehab
Dr. Ahmed Abdelfattah

CSEN 401 - Computer Programming Lab, *Spring 2026*

DooR DasH: Scare vs Laugh Touchdown

Milestone 2

Deadline: 1.5.2026 @ 11:59 PM

This milestone is a further *exercise* on the concepts of **object oriented programming (OOP)**. By the end of this milestone, you should have a working game engine with all its logic, that can be played on the console if needed. The following sections describe the requirements of the milestone. Refer to the **Game Description Document** for more details about the rules.

- In this milestone, you must adhere to the class hierarchy established in Milestone 1. You are not permitted to alter the visibility or access modifiers of any variables, unless such changes are explicitly mentioned in the description of this milestone. Additionally, you should follow the method signatures given in this milestone. However, you have the freedom to introduce additional helper methods as needed.
- All methods mentioned in the document should be `public` unless otherwise specified. All class attributes should be `private` with the appropriate access modifiers and naming conventions for the getters and setters as needed.
- You should always adhere to the OOP features when possible. For example, always use the `super` method or constructor in subclasses when possible.
- The model answer for M1 is available on CMS. It is recommended to use this version. A full grade in M1 doesn't guarantee a 100 percent correct code, it just indicates that you completed all the requirements successfully :)
- Some methods in this milestone depend on other methods. If these methods are not implemented or not working properly, this will affect the functionality and the grade of any method that depends on them.
- **You need to carefully read the entire document to get an overview of the game flow as well as the milestone deliverables, before starting the implementation.**
- You need to think about the boundary values of all variables. For example, you need to make sure that certain variables don't fall below zero
- Before any action is committed, the validity of the action should be checked. Some actions will not be possible if they violate game rules. **All exceptions that could arise from invalid actions should be thrown in this milestone.** You can refer to milestone 1 description document for the description of each exception and when it can occur.

Always refer back to the **Game Description Document** to clarify any misunderstood logic before proceeding with your implementation.

1 Interfaces

1.1 CanisterModifier Interface

1.1.1 Methods

This interface should include the following method:

1. `void modifyCanisterEnergy(Monster monster, int canisterValue)`: A method that modifies the energy of the given monster by the given `canisterValue`. The exact modification behaviour is determined by each implementing class.

2 Classes

2.1 Monsters Class

2.1.1 Methods

The `Monster` base class should include the following methods:

1. `abstract void executePowerupEffect(Monster opponentMonster)`: A method that executes unique powerup effects on opponent or all monsters. Each subclass provides its own implementation.
2. `boolean isConfused()`: A method that returns `true` if the monster currently has a non-zero confusion turn count.
3. `void move(int distance)`: A method that shifts the monster's original position by the input distance.
4. `void alterEnergy(int energy)`: A method that tries to set the monster's energy by the given amount respecting the shield. If the monster is shielded and the change is negative, the shield is consumed and the energy is not altered. Otherwise, the energy is **set** with the old value plus the incoming energy. This method is only used when shield is respected. Otherwise, `setEnergy(int energy)` is used. This method should not be overriden by any subclass.
5. `void decrementConfusion()`: A method that decrements the confusion turn counter by 1 if it is greater than zero. Once the counter reaches zero, the monster's role is restored to its original role.

The `Schemer` class should include the following method:

1. `private int stealEnergyFrom(Monster target)`: A helper method that gets `Constants.SCHEMER_STEAL` energy from the given target, or the target's full remaining energy if it falls below that amount. It should return the amount of energy stolen. n.b the player doesn't get the value just yet as he gets the total at once.

For each subclass, distinct behaviors can be implemented by overriding relevant methods if needed to suit their specific functionalities. Consider the special features of each monster below.
Note: setters receive the new value to be set, not the change — keep this in mind when overriding.

Monster's Types	Powerup Effect	Passive Features
Dynamo	Screech Freeze: Freezes the opponent for one turn.	Doubles all incoming energy **changes**, whether positive or negative.
Dasher	Momentum Rush: moves at 3× speed (distance) for 3 turns instead of the usual 2×.	Moves at 2× the dice roll.
MultiTasker	Focus Mode: moves at normal speed for 2 turns.	Moves at $\frac{1}{2}$ the dice roll. Gains +200 energy on all incoming energy **changes**, whether positive or negative.
Schemer	Chain Attack: steals energy from the opponent and all stationed monsters, gaining a single total steal bonus at the end.	Gains +10 energy on every incoming energy **changes**, whether positive or negative.

2.2 Cards Class

2.2.1 Methods

The `Card` base class should include the following method:

1. `abstract void performAction(Monster player, Monster opponent)`: A method that executes the card's unique effect on the player and/or opponent. Each subclass provides its own implementation.

For each subclass, distinct behaviors can be implemented by overriding `performAction` to suit their specific functionalities. Consider the special features of each card below.

Card Type	Effect
SwapperCard	If the player is behind the opponent in position, the two monsters swap their positions.
EnergyStealCard	Steals the card's energy amount from the opponent (or all their energy if less). Respects the shield mechanic. Player gains the actual stolen amount even if less than the card's value. **Modifies canister energy** of both the player and opponent.
ShieldCard	Grants the player a shield that blocks the next negative energy effect. Removes any existing shield on the opponent.
StartOverCard	Sends either the player or the opponent back to position 0, depending on the card's `lucky` flag.
ConfusionCard	Swaps the roles of **both** monsters for a set number of turns (`duration`). Causes door cells to award/penalise them in reverse.

2.3 Cells Class

2.3.1 Methods

The `Cell` base class should include the following methods:

1. `boolean isOccupied()`: A method that returns `true` if the cell currently has a landing monster on it, and `false` otherwise.
2. `void onLand(Monster landingMonster, Monster opponentMonster)`: A method that executes the cell's unique effect when a monster lands on it. A cell should keep track of the landing monster by setting its attributes.

The `TransportCell` subclass should include the following method:

1. `void transport(Monster monster)`: A method that applies the cell's transport effect to the given monster. Subclasses of `TransportCell` should override this method accordingly if needed.

For each subclass, distinct behaviors can be implemented by overriding `onLand` to suit their specific functionalities. Consider the special features of each cell below.

Cell Type	Effect on Landing
CardCell	Draws a card from the deck and performs its action.
DoorCell	If not yet activated, **modiefies canister energy** of the landing monster and stationed monsters of the same role as landing monster. Respects shield for both landing monster and team of stationed monsters. **Modifies canister energy** depending on whose side the door is on. Marks itself as activated if energy was gained or lost only.
ConveyorBelt	**Transports** the landing monster by the cell's **positive** `effect` value.
ContaminationSock	**Transports** the landing monster by the cell's **negative** `effect` value. **Modifies canister energy** of landing monster with `Constants.SLIP_PENALTY`.
MonsterCell	If the stationed monster shares the same role as the landing monster, executes the landing monster's powerup effect for free. Otherwise, if the landing monster has more energy than the stationed monster, the two monsters swap their energy values. Shield can block the energy penalty to landing monster only but cell monster still get their increased energy.

Game Setup

2.4 Board Class

Name : `Board`

Package : `game.engine`

Type : Class

Description : A class representing the 10×10 game board. Manages the cell layout, stationed monsters, and the card deck.

2.4.1 Methods

The `Board` class should include the following methods:

1. `private int[] indexToRowCol(int index)`: A method that converts a linear board index to a row and column pair, accounting for the zizag layout where even rows go left-to-right and odd rows go right-to-left (refer back to Indices Distribution map in game description). This method returns `int[]{row,col}`.
2. `private Cell getCell(int index)`: A method that returns the cell at the given board index using `indexToRowCol`.
3. `private void setCell(int index, Cell cell)`: A method that sets the cell at the given board index using `indexToRowCol`.
4. `void initializeBoard(ArrayList<Cell> specialCells)`: A method that populates the board grid using the special cells loaded from the CSV (includes different doors and Transport Cells). Even-indexed positions become normal rest cells and odd-indexed positions become `DoorCell`s. Card cells, conveyor belts, contamination socks, and monster cells are placed at their designated indices as defined in `Constants` in order. Stationed monsters are assigned positions and placed on the board.
5. `private void setCardsByRarity()`: A method that expands the original cards list by duplicating each card according to its rarity value, then replaces the original list with the expanded one.
6. `static void reloadCards()`: A method that resets the active card deck to a freshly shuffled copy of the original expanded card list.

7. `static Card drawCard()`: A method that removes and returns the top card (first index) from the shuffled deck. If the deck is empty, it is reloaded before drawing.

8. `void moveMonster(Monster currentMonster, int roll, Monster opponentMonster) throws InvalidMoveException`: Performs a full movement action for the current monster given a dice roll. The monster is moved by the roll value, and the landed cell's effect (like transport, card withdrawal, etc) is triggered via `onLand`. If the final resulting position after landing collides with the opponent's position, the monster is reverted to its old position and an `InvalidMoveException` is thrown. If monsters are confused, confusion is decremented for both monsters after landing on a cell (n.b confusion shouldn't be decremented immediately upon receiving it). Finally, note that actions like swapping can occur with `onLand`, changing both player and opponent monsters' positions while leaving the board's cell references out of sync with the monsters' actual positions. Therefore, the board's cell references are synchronised with monsters' positions via `updateMonsterPositions`.

9. `private void updateMonsterPositions(Monster player, Monster opponent)`: A helper method that synchronises the board's cell-monster references with the current positions of both monsters. Clears all cell monster references across the board, then reassigns each monster to its correct cell based on their current position.

2.4.2 Constructors

The `Board` constructor from Milestone 1 should now call the following methods:

1. `setCardsByRarity()` to expand the card list by rarity.
2. `reloadCards()` to populate and shuffle the active card deck.

2.5 Game Class

Name : `Game`

Package : `game.engine`

Type : Class

Description : A class representing the game itself. This class is the main engine that manages the overall flow of the game including turns, powerups, and the win condition.

2.5.1 Methods

The `Game` class should include the following methods:

1. `private Monster getCurrentOpponent()`: A method that returns the monster that is not the current active monster.
2. `private int rollDice()`: A method that returns a uniformly random integer between 1 and 6 inclusive.
3. `void usePowerup() throws OutOfEnergyException`: A method that activates the current monster's powerup if the current monster has sufficient energy (`Constants.POWERUP_COST`). Throws an `OutOfEnergyException` otherwise.
4. `void playTurn() throws InvalidMoveException`: A method that performs one full turn for the current monster. If the monster is frozen, its turn is skipped and it is unfrozen. Otherwise, the dice is rolled and the monster is moved on the board. The turn is then switched to the other monster.
5. `private void switchTurn()`: A method that transfers turn to the other monster.
6. `private boolean checkWinCondition(Monster monster)`: A method that evaluates whether the given monster satisfies all conditions required to win the game.

7. `Monster getWinner()`: A method that determines if either monster has met the win condition. Returns the winning monster if one exists, otherwise returns `null`.

2.5.2 Constructors

The `Game` constructor from Milestone 1 should now call the following methods from the `Board` class:

1. `setStationedMonsters(ArrayList<Monster> stationedMonsters)` to assign the remaining monsters to the board. (without the player and opponent)
2. `initializeBoard(ArrayList<Cell> specialCells)` to set up the board cells using the loaded cell data from CSV.